

Introduction to Simulators

- **What is a Simulator?** A software environment that mimics the physical world, allowing robots to be tested virtually.
- **Why use them?**
 - **Safety:** No risk of breaking expensive hardware.
 - **Speed:** Can run faster than real-time.
 - **Cost:** Free to create complex scenarios.
- **Key Components:**
 - **Rendering Engine:** Visual output.
 - **Physics Engine:** Dynamics & Collisions.
 - **ROS Interface:** Bridge to ROS 2.



Physics Engines: The Math Behind Simulation

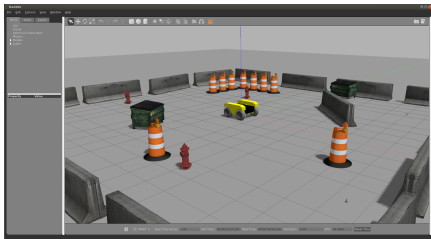
- **Simulator vs. Engine:** A simulator provides the UI and rendering. The physics engine solves the math.
- **Common Physics Engines:**
 - **ODE:** Classic, used in older Gazebo.
 - **Bullet / PyBullet:** Popular in RL and robotics.
 - **DART:** Focuses on articulated dynamics.
 - **MuJoCo Engine:** Optimized for contact dynamics.



Gazebo: The ROS Standard

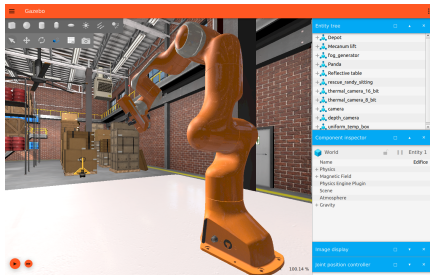
Gazebo Classic (ROS 1 / early ROS 2)

- Monolithic architecture.
- Uses ODE by default.
- Simple rendering (sufficient for navigation).



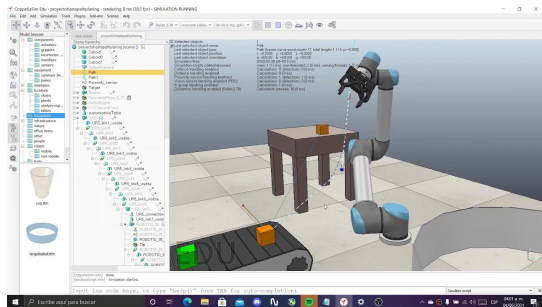
Modern Gazebo (Ignition / Harmonic)

- Modular architecture (plugins).
- Supports multiple physics engines (DART, Bullet).
- Improved rendering (PBR).



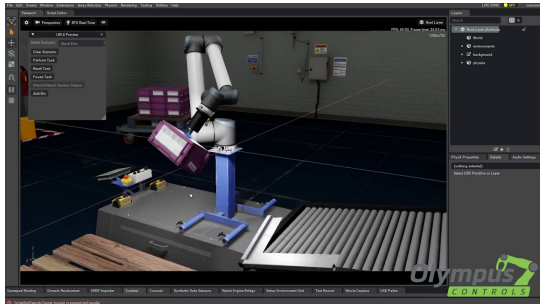
CoppeliaSim (formerly V-REP)

- **Flexibility:** Highly scriptable using Lua, Python, C++, or MATLAB.
- **Assets:** Comes with a massive library of pre-built robots, sensors, and environments.
- **Engines:** Supports Bullet, ODE, Newton, and Vortex.
- **Use Case:** Rapid prototyping and factory automation simulation.



Isaac Sim: Photorealism and GPU Power

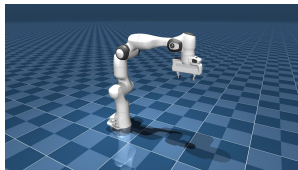
- **Engine:** NVIDIA PhysX (GPU accelerated).
- **Photorealism:** Ray tracing and advanced lighting.
- **Why is photorealism important?**
 - Generating synthetic data for Computer Vision.
 - Sim2Real transfer for Reinforcement Learning (RL).
 - If the sim looks real, the AI performs better in reality.



MuJoCo and ManiSkill: RL and Manipulation

MuJoCo

- Exceptional at handling complex contact dynamics.
- Extremely fast CPU performance.
- Standard for continuous control RL tasks.



ManiSkill

- Built on SAPIEN (Vulkan-based).
- Specialized for robotic manipulation tasks.
- Massive GPU parallelization for RL





Chapter 1 References

- [Gazebo Official Website](#)
- [MuJoCo Physics Engine](#)
- [ManiSkill Project Page](#)
- [NVIDIA Isaac Sim](#)
- [CoppeliaSim Official Website](#)



Unified Robot Description Format (URDF)

- **Definition:** XML-based format describing the physical structure of a robot in ROS.
- **Purpose:** Centralizes kinematics, visual appearance, and collision properties.
- **Core Components:**
 - **Links:** Rigid bodies (e.g., chassis, wheels) with inertia, visual, and collision data.
 - **Joints:** Connections between links (revolute, continuous, fixed, etc.).
- **Xacro:** XML Macros to make URDF modular, reusable, and parameterized.

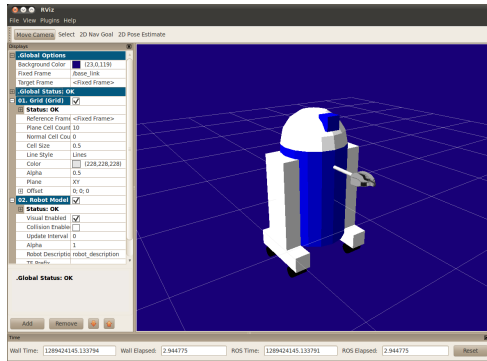


URDF Code Example

```
<robot name="my_robot">
  <link name="gripper_pole">
    <visual>
      <geometry><cylinder length="0.2" radius="0.01"/>
      </geometry>
      <origin rpy="0 1.57075 0 " xyz="0.1 0 0"/>
    </visual>
  </link>

  <joint name="left_gripper_joint" type="fixed">
    <origin rpy="0 0 0" xyz="0.2 0.01 0"/>
    <parent link="gripper_pole"/>
    <child link="left_gripper"/>
  </joint>

  <link name="left_gripper">
    <visual><origin rpy="0.0 0 0" xyz="0 0 0"/>
      <geometry>
        <mesh filename="package://urdf_tutorial/meshes/l_finger.dae"/>
      </geometry>
    </visual></link>
</robot>
```





Chapter 2 References

- ROS 2 URDF Tutorials
- SDF Specification
- MuJoCo MJCF Modeling Guide



The Role of Coordinate Frames (TF2)

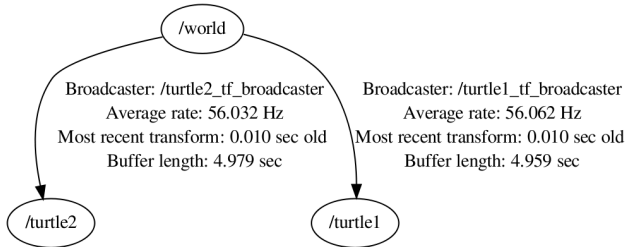
- **Concept:** Robots operate with multiple coordinate systems (frames). TF tracks these frames over time.
- **Structure:** A tree hierarchy where every frame has exactly one parent (no closed loops).
- **Types of Transforms:**
 - **Static:** Constant over time (e.g., sensor bolted to chassis). Published to `/tf_static`.
 - **Dynamic:** Continuously updated (e.g., moving wheels, robot in map). Published to `/tf`.
- **robot_state_publisher:** Node that reads URDF and joint states to publish the internal TF tree.



TF Tree Example

view_frames Result

Recorded at time: 1254266629.492





TF and Sensor Data (std_msgs/Header)

- Sensor data (e.g., LaserScan, Image) must be linked to the TF tree.
- This is done via the std_msgs/Header message type.
- **frame_id**: Specifies which TF frame the data belongs to.
- **stamp**: The exact time the data was captured (crucial for dynamic transforms).

```
# Example of a Header in a ROS 2 message
```

```
header:
```

```
  stamp:
```

```
    sec: 1623456789
```

```
    nanosec: 123456789
```

```
  frame_id: 'lidar_link'
```



TF CLI Commands

- **Publish a static transform manually:**

```
ros2 run tf2_ros static_transform_publisher x y z yaw pitch roll frame_id child_frame_id  
ros2 run tf2_ros static_transform_publisher 0.1 0 0.2 0 0 0 base_link camera_link
```

- **View the current TF tree (generates a PDF):**

```
ros2 run tf2_tools view_frames
```

- **Print the transform between two frames:**

```
ros2 run tf2_ros tf2_echo map base_link
```



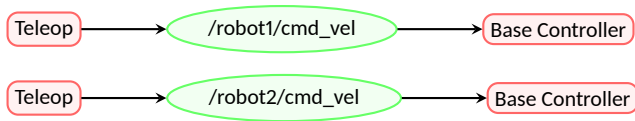
Chapter 3 References

- Introduction to tf2
- REP-105: Coordinate Frames standard



Namespace Isolation

- **Problem:** Multiple robots in one network will publish to the same topics (e.g., /cmd_vel), causing conflicts.
- **Solution:** Namespaces group nodes, topics, and services under a specific prefix.
- **Result:** /robot1/cmd_vel is completely isolated from /robot2/cmd_vel.





Using Namespaces via CLI

- You can push a node into a namespace directly from the command line using `--ros-args -r __ns:=<namespace>`.

```
# Run a talker node in the /robot1 namespace
ros2 run demo_nodes_cpp talker --ros-args -r __ns:=/robot1

# Check the topics
ros2 topic list

# Output will include: /robot1/chatter
```

- In launch files, the `PushRosNamespace` action is used to apply a namespace to an entire group of nodes.



Namespaces and the TF Problem

- **The TF Exception:** By default, `/tf` and `/tf_static` are **global** topics. They are usually NOT namespaced.
- **Why?** The TF tree is designed to be a single, unified graph for the entire system.
- **Namespacing TF:** You *can* push TF into a namespace (e.g., `/robot1/tf`), but it causes problems:
 - Standard tools (like RViz) expect a global `/tf` topic.
 - In code, `tf2_ros::TransformListener` uses a special mechanism to subscribe to TF. It does not read TF like a normal topic.
 - If TF is namespaced, you must explicitly remap the listener's topics or configure it to use the specific namespace, which is error-prone.
- **Best Practice:** Keep `/tf` global, but prefix the `frame_ids` (e.g., `robot1/base_link`).



Chapter 4 References

- ROS 2 Nodes and Namespaces
- Nav2 Multi-Robot Setup Guide

FastDDS vs. CycloneDDS

FastDDS (eProsima)

- Default in ROS 2 Foxy/Humble.
- **Pros:** Out-of-the-box Shared Memory (SHM) for zero-copy local transfers. Discovery Server feature solves Wi-Fi issues.
- **Cons:** Default multicast discovery floods Wi-Fi networks. Complex XML configuration.
- **Verdict:** Use **CycloneDDS** for stable multi-robot Wi-Fi setups. Use **FastDDS** for high-performance single-machine systems (cameras/lidars) or complex WANs.

CycloneDDS (Eclipse)

- Default in ROS 2 Galactic.
- **Pros:** Excellent "out-of-the-box" stability on lossy networks (Wi-Fi). Simple configuration. Lightweight.
- **Cons:** Lacks native zero-copy SHM (requires iceoryx integration). Less performant for massive local payloads.

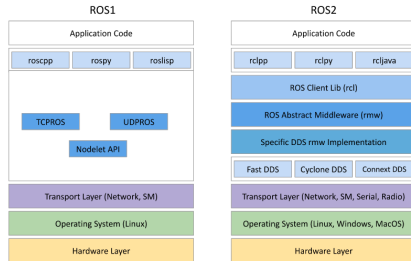


DDS (Data Distribution Service)

- **Concept:** Data-centric, peer-to-peer protocol with rich QoS settings.
- **FastDDS:** Default in ROS 2 Humble.
- **CycloneDDS:** Often considered more stable "out of the box".
- **Switching DDS:**

```
# Install CycloneDDS
sudo apt install ros-humble-rmw-cyclonedds-cpp

# Switch implementation
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```



Zenoh: Bridging the WAN Gap

- **The Problem:** DDS relies on multicast for discovery, which fails over Wi-Fi, VPNs, or Wide Area Networks (WAN).
- **Zenoh:** A "Zero Overhead" protocol designed for efficiency and scale.
- **zenoh-plugin-dds:** Acts as a bridge, transparently routing DDS traffic across a Zenoh network (e.g., Robot to Cloud).





Chapter 5 References

- [Zenoh Official Website](#)
- [Zenoh DDS Plugin GitHub](#)
- [OMG DDS Specification](#)



Why Test in Robotics?

- **Safety:** Prevent regressions that could damage hardware.
- **CI/CD Integration:** Automatically validate code on every commit.
- **Edge Cases:** Safely test dangerous scenarios in simulation.
- **Benchmarking:** Quantitatively measure performance metrics.



Levels of Testing in ROS 2

- **Unit Tests (GTest / Pytest):** Test individual functions or classes in isolation.
- **Integration Tests (launch_testing):** Test the interaction between multiple ROS nodes (e.g., does Node A correctly process messages from Node B?).
- **System / SITL Tests:** "Simulation-in-the-loop". Run the entire software stack against a physics simulator to validate complex behaviors.

```
# Build tests
colcon build --packages-select my_package --cmake-args -DBUILD_TESTING=ON

# Run tests
colcon test --packages-select my_package

# View results
colcon test-result --all
```



Chapter 6 References

- [ROS 2 Testing Guide](#)
- [Colcon Testing Documentation](#)

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521